K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

ENSP465 — New Age Programming Languages Lab

Capstone Project — Phase I: Project Proposal & System Design

TaskFlow

A Distributed Job Scheduler in Go

Submitted by

S.No.	Student Name	Roll Number
1	Adeela Azeez	2301010282
2	Archi Garg	2301010346
3	Kirti Sahni	2301010236
4	Abhishek Raj	2301010111
5	Vinay Pratap Singh	2301010231

Semester: VII | Academic Year: 2026–27

Submission Date: 24 October 2026

Submitted to: Dr. Hemraj Kumawat

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

Sohna, Haryana 122103, India

2. Problem Statement & Motivation

2.1 Problem Description

Almost every modern software system needs to run work in the background: sending emails, generating reports, processing payments, resizing images, syncing data with third-party services, or cleaning up stale records. This work should not block the user-facing request, often needs to run at a specific time or on a recurring schedule, and must complete reliably even when individual machines fail.

The traditional answer is a cron job on a single server. Cron has no notion of distribution, priority, retries, timeouts, or visibility. If the server crashes, scheduled jobs silently do not run; if a job hangs, nothing stops it; if a job fails, nobody is told and nothing is retried. As an application grows to multiple servers, teams end up writing ad-hoc locking and retry code that is fragile and hard to observe.

TaskFlow addresses this gap: a distributed job scheduler that lets users submit, schedule, prioritise and monitor background jobs, executes them across a pool of worker nodes, and guarantees that failed, timed-out, or orphaned jobs are detected and recovered automatically.

2.2 Real-World Context

Domain: backend infrastructure / distributed systems (web service + worker cluster). Typical users are backend developers and operations teams at product companies — for example, a fintech platform that must run nightly settlement reconciliation, retry failed payment webhooks, and send scheduled statements without losing a single job.

2.3 Motivation

- Reliability: lost or duplicated background jobs translate directly into lost revenue or inconsistent data.
- Observability: teams need to see what is queued, running, failed and retried in real time rather than grepping logs.
- Learning value: the project exercises core distributed-systems concepts — concurrency, queues, leases, heartbeats, idempotency and failure recovery — which map naturally onto Go’s concurrency model.

2.4 Existing Solutions and Their Limitations

Tool	What it does well	Limitation for our use case
Unix cron	Simple time-based scheduling	Single machine; no retries, priorities, timeouts or dashboard
Celery (Python)	Mature distributed task queue	Python-only; heavy configuration; limited built-in monitoring
Sidekiq / BullMQ	Fast Redis-backed queues	Tied to Ruby / Node.js ecosystems; advanced features are paid (Sidekiq Pro)
Apache Airflow	Complex DAG workflows	Heavyweight for simple jobs; high operational overhead
Kubernetes CronJob	Cluster-native scheduling	Requires a Kubernetes cluster; coarse visibility; slow pod start-up per job

TaskFlow targets the middle ground: lightweight enough to run as a few Go binaries with Redis and PostgreSQL, yet providing priorities, retries, timeouts, crash recovery and live monitoring out of the box.

3. Proposed Solution & Language Justification

3.1 Project Overview

TaskFlow is a distributed job scheduling system in which users submit jobs (immediate, delayed, or recurring via cron expressions) through a REST API or a React dashboard. A scheduler places due jobs into Redis priority queues, a cluster of Go worker nodes executes them concurrently with enforced timeouts, and a recovery manager retries failures with exponential backoff and re-queues jobs from crashed workers — while every state change is streamed live to the dashboard.

3.2 Language Chosen

Primary implementation language: Go. All backend services — API server, scheduler, workers, recovery manager and WebSocket hub — are written in Go. React (JavaScript) is used only for the thin monitoring front-end.

3.3 Language Justification

1. **Goroutines for massive, cheap concurrency.** Each worker node runs a pool of goroutines (a few KB of stack each), so a single node can execute hundreds of jobs in parallel without the overhead of OS threads or a separate process per job.
2. **Channels and select for safe coordination.** Jobs are handed from the fetcher to the worker pool over buffered channels, giving natural back-pressure. select lets a worker wait simultaneously on a new job, a shutdown signal and a heartbeat ticker without shared-memory locking.

3. **context package for timeouts and cancellation.** context.WithTimeout gives every job a hard deadline that propagates into HTTP calls and database queries, so hung jobs are cancelled cleanly — a core requirement of the system.
4. **Static binaries and fast start-up.** Go compiles to a single dependency-free binary, making it trivial to start additional worker nodes (horizontal scaling) on any machine or container.
5. **Interfaces for pluggable job handlers.** A small JobHandler interface lets new job types be registered without modifying the core engine, keeping the system open for extension.

3.4 Key Libraries and Frameworks

Library / Tool	Purpose
go-chi/chi	Lightweight HTTP router for the REST API
redis/go-redis v9	Redis client: sorted-set priority queues, heartbeats, pub/sub
jackc/pgx v5	High-performance PostgreSQL driver and connection pool
robfig/cron v3	Parsing and evaluating cron expressions for recurring jobs
gorilla/websocket	Real-time status streaming to the dashboard
log/slog, testing	Structured logging and unit/integration tests (Go standard library)
React + Vite	Monitoring dashboard (front-end only)
Docker Compose	Local multi-node deployment of Redis, PostgreSQL and workers

4. Objectives & Scope

4.1 Objectives

1. Implement a concurrent worker engine in Go that executes at least 100 jobs in parallel across a minimum of 3 worker nodes.
2. Support 3 priority levels (high, normal, low) such that a high-priority job is dequeued before any waiting lower-priority job.
3. Automatically retry failed jobs up to a configurable limit (default 3) using exponential backoff, and move exhausted jobs to a dead-letter state.
4. Enforce per-job timeouts so that no job runs beyond its configured deadline, and detect a crashed worker and re-queue its jobs within 15 seconds of its last heartbeat.
5. Provide a dashboard that reflects job status changes in under 1 second via WebSockets.

4.2 In Scope (Phase II and III)

- REST API to create, list, cancel and inspect jobs and their run history.
- Immediate, delayed and cron-based recurring jobs.
- Redis-backed priority queues with atomic job claiming.
- Multi-node worker pool with heartbeats and graceful shutdown.

- Retries with exponential backoff, timeouts, dead-letter handling and crash recovery.
- PostgreSQL persistence of jobs, runs and logs.
- React dashboard with live updates; Docker Compose setup; unit, integration and load tests.

4.3 Out of Scope

- Multi-tenant user accounts, role-based access control and OAuth (a single static API key is used).
- DAG / workflow dependencies between jobs (job B after job A).
- Executing arbitrary user-uploaded code or shell scripts; only pre-registered job handlers run.
- Kubernetes auto-scaling, multi-region deployment and Redis/PostgreSQL high-availability clustering.
- Mobile application.

5. System Architecture & Design

5.1 High-Level Architecture

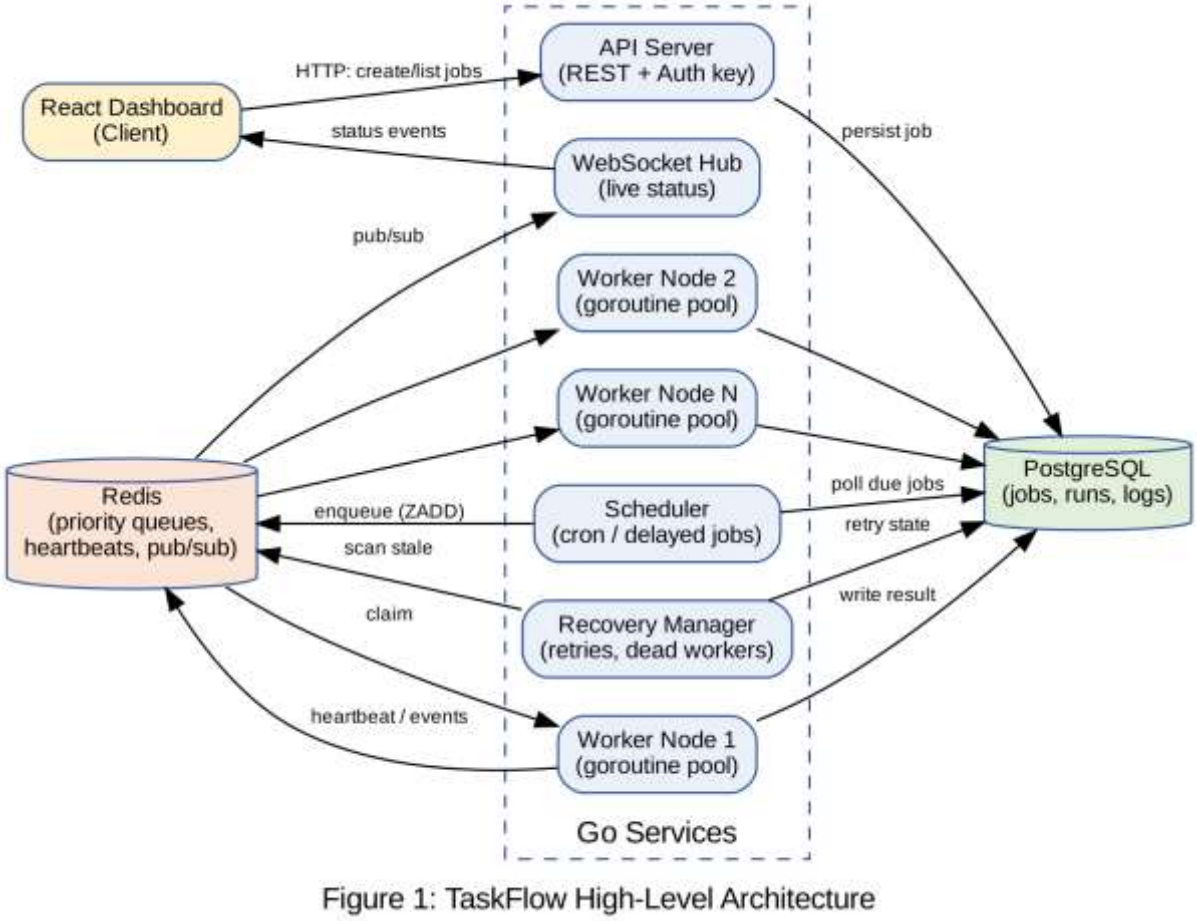


Figure 1: TaskFlow High-Level Architecture

Figure 1: High-level architecture of TaskFlow showing Go services, data stores and client.

5.2 Component Description

API Server. Accepts JSON job specifications over HTTP (type, payload, priority, schedule, timeout, max retries), validates them, stores them in PostgreSQL and returns a job ID. Also serves list, detail and cancel endpoints. Input: HTTP requests. Output: job records and JSON responses.

Scheduler. Runs on a ticker (every second), queries PostgreSQL for jobs whose next_run_at is due, and pushes their IDs into the Redis sorted set for their priority. For recurring jobs it computes the next run time from the cron expression. A Redis lock ensures only one scheduler instance enqueues at a time. Input: due job rows. Output: queued job IDs.

Worker Nodes. Each node runs a fetcher goroutine that atomically claims the highest-priority job from Redis and sends it on a buffered channel to a pool of N executor goroutines. Each execution is wrapped in context.WithTimeout. Nodes publish heartbeats every 5 seconds. Input: claimed job. Output: result, error or timeout, written to PostgreSQL and published as an event.

Recovery Manager. Handles failed jobs by computing a backoff delay (base × 2^attempt) and re-scheduling them, or marking them dead after max retries. It also scans for workers whose heartbeat has expired and re-queues their in-progress jobs. Input: failure events, heartbeat keys. Output: re-queued jobs and updated status.

WebSocket Hub. Subscribes to the Redis pub/sub channel of job events and fans them out to all connected dashboard clients. Input: job events. Output: live JSON messages to the browser.

PostgreSQL. Source of truth for jobs, run attempts (start time, end time, status, error) and logs.

Redis. Holds the priority queues (sorted sets), in-progress sets, worker heartbeats (keys with TTL), distributed locks and the pub/sub event channel.

React Dashboard. Lets users submit jobs, view queues and worker health, and inspect job history, updating live via WebSocket.

5.3 Data Flow Diagram (Level 1)

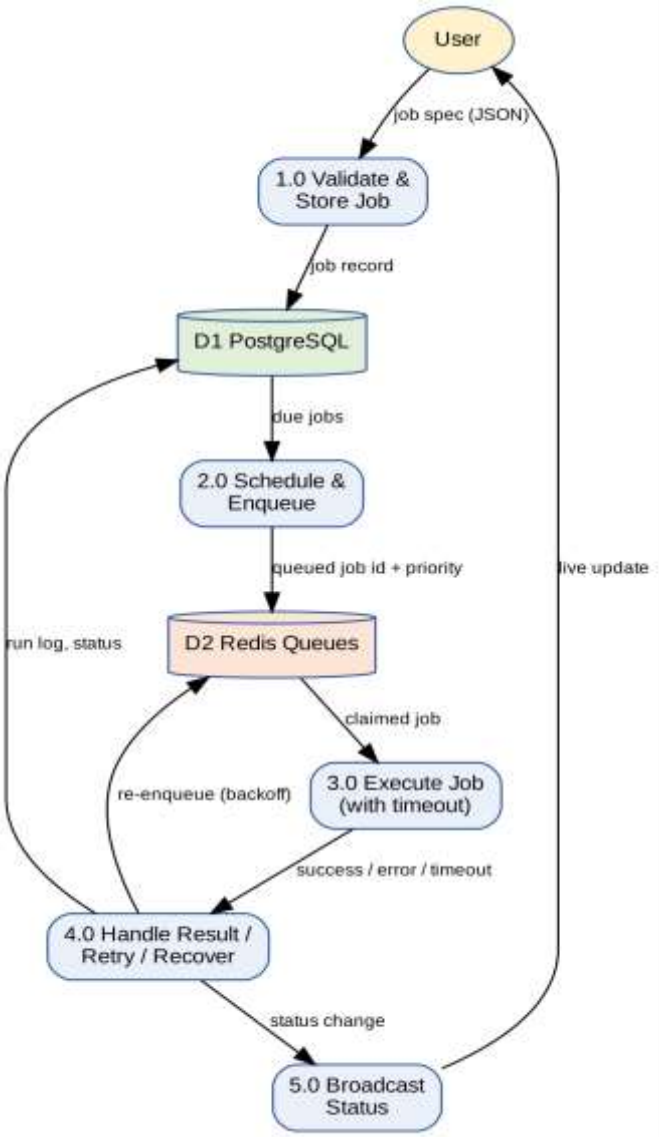


Figure 2: DFD Level 1 showing the lifecycle of a job from submission to completion.

5.4 Main Workflow (Sequence)

1. The user submits a job (JSON) via the dashboard or API.
2. The API server validates the request, inserts the job with status PENDING into PostgreSQL and returns the job ID.
3. The scheduler finds the job due, adds its ID to the matching Redis priority queue and sets status QUEUED.
4. A worker's fetcher atomically moves the job to its in-progress set (status RUNNING) and passes it to an executor goroutine.
5. The executor runs the registered handler under a timeout context while the node keeps sending heartbeats.
6. On success the result is stored (SUCCEEDED). On error or timeout the recovery manager schedules a retry with backoff (RETRYING) or marks it DEAD after max retries.

7. Every status change is published to Redis pub/sub and pushed to the dashboard by the WebSocket hub.

5.5 Input–Process–Output (IPO)

Input	Process	Output
Job specification: type, payload (JSON), priority, schedule (now / delay / cron), timeout, max retries	Validate → persist → schedule → enqueue by priority → claim → execute concurrently with timeout → retry / recover on failure	Job status (PENDING → QUEUED → RUNNING → SUCCEEDED / RETRYING / DEAD), result data, run history
Worker heartbeats	Detect stale workers and re-queue their orphaned jobs	Recovered jobs, worker health metrics
Dashboard queries	Aggregate queue depth, throughput and failure counts	Live dashboard views and REST responses

5.6 Key Data Structures and Go Features

Go feature / structure	Use in TaskFlow
Structs with JSON tags	Job, JobRun and WorkerInfo types shared between API, database and queue layers
Goroutines	Executor pool, fetcher, heartbeat sender, scheduler ticker and per-client WebSocket writers
Buffered channels	Job hand-off from fetcher to executors with built-in back-pressure; results channel
select statement	Waiting on jobs, ctx.Done() for shutdown, and time.Ticker for heartbeats at once
context.Context	Per-job timeouts, cancellation of user-cancelled jobs and graceful shutdown
sync.WaitGroup / sync.Mutex	Draining in-flight jobs on shutdown; guarding the WebSocket client registry
Interfaces	JobHandler { Handle(ctx, payload) error } and a Queue interface to allow mocking in tests
Map registry	map[string]JobHandler mapping job type names to their handlers
Redis sorted sets	Priority queues scored by priority and enqueue time; Lua script for atomic claim
Error wrapping (errors.Is/As)	Distinguishing retryable errors from permanent failures

6. Work Plan & Timeline

6.1 Milestones

#	Milestone	Deliverables	Tentative Dates
M1	Phase I: Proposal & Design	This document; architecture, DFD, data model	24 Oct 2026
M2	Core setup & persistence	Go module layout, Docker Compose (Redis, PostgreSQL), DB schema & migrations, REST API skeleton	25 Oct – 2 Nov 2026
M3	Queue, scheduler & workers	Priority queues, scheduler with cron support, worker pool with timeouts and heartbeats	3 Nov – 12 Nov 2026
M4	Phase II: Intermediate Demo	End-to-end flow: submit → schedule → execute on multiple workers; basic dashboard	Mid-Nov 2026 (as per schedule)
M5	Reliability & real-time UI	Retries with backoff, dead-letter, crash recovery, WebSocket live updates, full dashboard	15 Nov – 25 Nov 2026
M6	Testing & documentation	Unit/integration tests, load test (100+ parallel jobs, worker-kill test), README, final report	26 Nov – 3 Dec 2026
M7	Phase III: Final Submission & Viva	Final deliverable, demo and individual viva	Early Dec 2026 (as per schedule)

Phase II deliverable: a working pipeline in which jobs submitted through the API are scheduled, prioritised and executed concurrently across multiple worker nodes, with results stored in PostgreSQL and visible on a basic dashboard.

Phase III deliverable: the complete system including retries, timeouts, crash recovery and live dashboard, plus automated tests, load-test results, documentation and a live demonstration for the viva.

6.2 Task Distribution

Member	Roll Number	Responsibility
Adeela Azeez	2301010282	Team lead; overall architecture; worker engine (goroutine pool, timeouts, heartbeats); integration
Archi Garg	2301010346	Scheduler and Redis priority queues (cron parsing, atomic claim, distributed lock)
Kirti Sahni	2301010236	REST API server and PostgreSQL schema / data-access layer
Abhishek Raj	2301010111	Recovery manager (retries, backoff, dead-letter, crash recovery) and testing / load testing
Vinay Pratap Singh	2301010231	React dashboard, WebSocket hub, Docker Compose setup and documentation

7. References

[1] The Go Authors, “The Go Programming Language Documentation.” [Online]. Available: https://go.dev/doc/ (Accessed: Oct. 2026).

[2] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[3] K. Cox-Buday, Concurrency in Go: Tools and Techniques for Developers. Sebastopol, CA, USA: O’Reilly Media, 2017.

[4] The Go Authors, “Package context.” [Online]. Available: https://pkg.go.dev/context (Accessed: Oct. 2026).

[5] Redis Ltd., “Redis Documentation — Sorted Sets and Pub/Sub.” [Online]. Available: https://redis.io/docs/ (Accessed: Oct. 2026).

[6] The PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/ (Accessed: Oct. 2026).

[7] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O’Reilly Media, 2017.

[8] hibiken, “Asynq: Simple, reliable, and efficient distributed task queue in Go,” GitHub repository. [Online]. Available: https://github.com/hibiken/asynq (Accessed: Oct. 2026).

[9] robfig, “cron: a cron library for Go,” GitHub repository. [Online]. Available: https://github.com/robfig/cron (Accessed: Oct. 2026).